



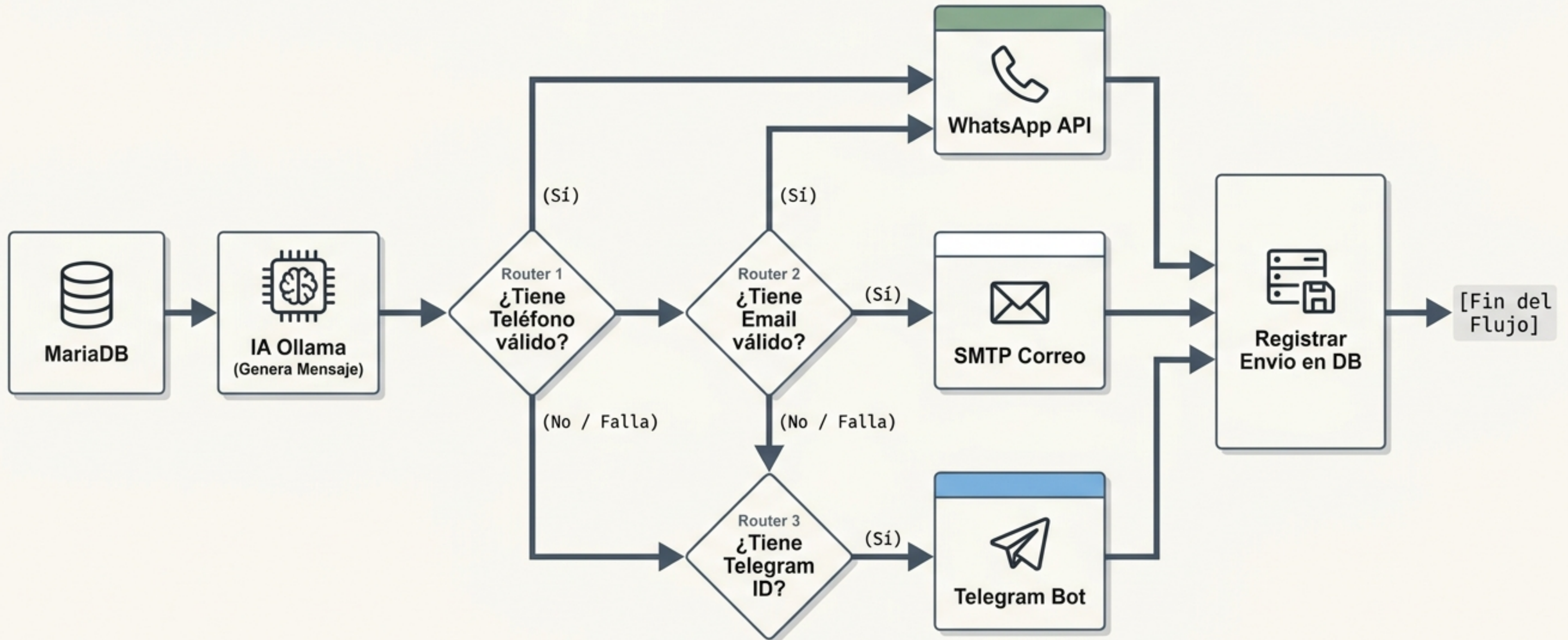
# Gestión Automatización de Cartera y Cobros

Presentado por: Yeferson Florez y Mateo Mendivelso

**MATERIA: INFRAESTRUCTURA DE SISTEMAS OPERATIVOS**



# Arquitectura de Fallbacks: Resiliencia multicanal con procesamiento local.





# Nodo 1: "Manual Trigger" (Disparador Manual)



**Tipo:** `n8n-nodes-base.manualTrigger`

## Propósito:

Permite ejecutar el **flujo completo a demanda** con **un solo clic** desde la interfaz **web de n8n**. Es **indispensable** para realizar **pruebas controladas** en clase sin tener **que esperar** a que se cumpla un horario programado.

## Configuración Clave:

No requiere parámetros especiales. Se activa con el botón 'Execute Workflow'.



## Nodo 2: "Schedule Trigger" (Disparador Programado - Cron)



**Tipo:** `n8n-nodes-base.cron`

### Propósito:

**Automatiza** la ejecución periódica del flujo en producción. Está configurado para correr de forma autónoma **una vez al día** por la mañana.

### Configuración Clave:

Modos de ejecución basados en tiempo (Ej. diario a las 8:00 AM). Garantiza que la revisión de cartera vencida sea un proceso desatendido.



## Nodo 3: "Consultar Facturas Vencidas" (Consulta a Base de Datos MySQL)



**Tipo:** n8n-nodes-base.mysql

**Propósito:** Conectarse a la base de datos MariaDB (**cartera\_db**) instalada en la VPS y extraer las facturas que tengan deudas sin pagar y cuya fecha límite de pago ya haya expirado.

Credentials: Usa la credencial 'MariaDB Cartera' (host: localhost/IP, puerto: 3306).

Operation: executeQuery (Permite escribir sentencias SQL puras).

SQL Query: `SELECT id, cliente_nombre, cliente_email, cliente_telefono, cliente_telegram_id, monto, fecha_vencimiento FROM facturas WHERE estado = 'pendiente' AND fecha_vencimiento < CURDATE()`

### Explicación del SQL:

`estado = 'pendiente'`: Filtra solo las facturas que no han sido pagadas.

`fecha_vencimiento < CURDATE()`: Selecciona solo los registros donde la fecha de vencimiento es estrictamente menor a la fecha actual (facturas vencidas).



## Nodo 4: 'Generar Mensaje con IA' (Integración con Ollama)



**Tipo:** @n8n/n8n-nodes-langchain.ollama

**Propósito:** Generar de manera dinámica un mensaje persuasivo y personalizado para cada cliente. En lugar de enviar un texto genérico, la IA redacta el mensaje según el nombre del cliente y el monto adeudado.

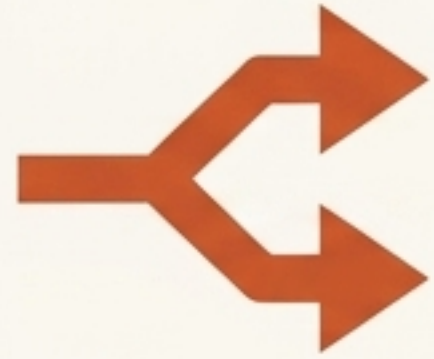
**Model:** Conectado al servicio local de Ollama que corre en la VPS (usando un modelo ligero y rápido como Llama 3 o similar).

**Prompt:** 'Eres un asistente de cartera muy amable. Genera un recordatorio corto y formal de pago para el cliente {{ \$json.cliente\_nombre }} por un monto de {{ \$json.monto }} USD que venció el día {{ \$json.fecha\_vencimiento }}. Sé breve y profesional.'

**Output:** El texto redactado por la IA se inyecta en el flujo como una nueva propiedad (text o content) para ser enviada por los nodos de mensajería.



## Nodo 5: “Tiene Teléfono?” (Bifurcación / Router IF)



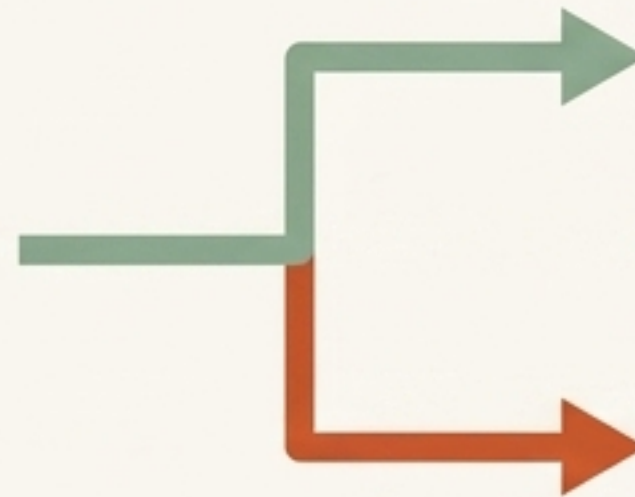
**Tipo:** n8n-nodes-base.if

**Propósito:** Determinar si el cliente tiene un canal de telefonía móvil registrado para intentar el envío prioritario por WhatsApp.

### Condición:

Compara el valor de  
`{{ $json.cliente_telefono }}`.

### Regla:



#### TRUE Path (WhatsApp)

Si la propiedad no está vacía (is not empty) y contiene un número con longitud razonable, toma el camino TRUE (hacia WhatsApp).

#### FALSE Path (Correo)

Si está vacía, toma el camino FALSE (hacia el envío de correo).



## Nodo 6: "WhatsApp (HTTP Request)" (Envío de Mensajes por WhatsApp)



**Tipo:** n8n-nodes-base.httpRequest

**Propósito:** Comunicarse con la API de 'Evolution API' instalada en la VPS para enviar el mensaje generado por la IA directamente al WhatsApp del cliente.

**Method:** POST

**URL:** `http://localhost:8085/message/sendText/mi_instancia`

**Headers:** apiKey de Evolution API para autorización.

**JSON Body:**

```
{ "number": "{{ $json.cliente_telefono }}", "text": "{{ $('Generar Mensaje con IA').item.json.text }}" }
```

Explicación de expresiones:

`{{ $json.cliente_telefono }}`: Lee el teléfono del cliente actual.

`{{ $('Generar Mensaje con IA').item.json.text }}`: Recupera el mensaje exacto redactado por el nodo de Inteligencia Artificial anterior.

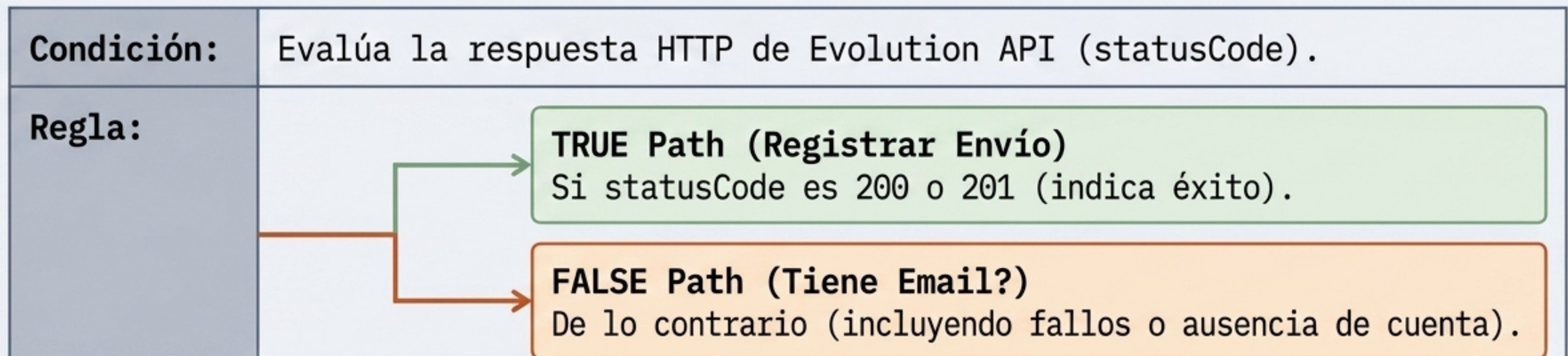


## Nodo 7: “¿WhatsApp Enviado?” (Verificación de Entrega)



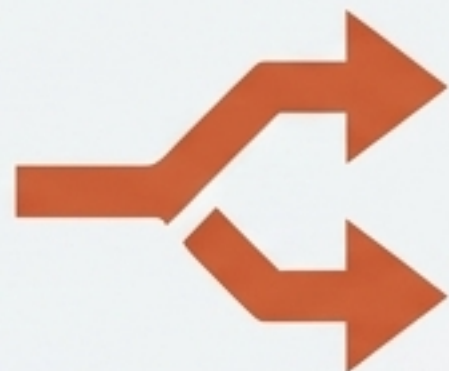
**Tipo:** n8n-nodes-base.if

**Propósito:** Evaluar si el envío por WhatsApp fue exitoso. Si el número no tiene WhatsApp activo o la API falló, n8n debe redirigir el mensaje al correo como alternativa (fallback).





# Nodo 8: “Tiene Email?” (Bifurcación / Router IF)



**Tipo:** n8n-nodes-base.if

**Propósito:** Validar si existe una dirección de correo registrada en el campo `cliente_email` para enviar la alerta por este medio.

## Configuración Clave:

### Regla:

Si `cliente_email` no está vacío y contiene el carácter '@', va a TRUE (Email Send).  
Si está vacío, va a FALSE (Tiene Telegram?).

TRUE Path (Email Send)

Si `cliente_email` no está vacío y contiene el carácter '@'

FALSE Path (Tiene Telegram?)

Si está vacío, va a FALSE (Tiene Telegram?)



## Nodo 9: 'Email Send' (Envío de Correo Electrónico por SMTP)



**Tipo:** `n8n-nodes-base.emailSend`

**Propósito:** Enviar una notificación formal de cobro al buzón de correo del cliente.

### Configuración Clave:

**Credentials:** Usa una cuenta SMTP (como Gmail o Outlook configurado en n8n).

**To:** `{{ $json.cliente_email }}`

**Subject:** `'Recordatorio Importante de Pago - {{ $json.cliente_nombre }}'`

**Body:** Inyecta el texto generado por la IA en formato texto o HTML.



## Nodo 10: '¿Email Enviado?' (Verificación de Entrega de Correo)

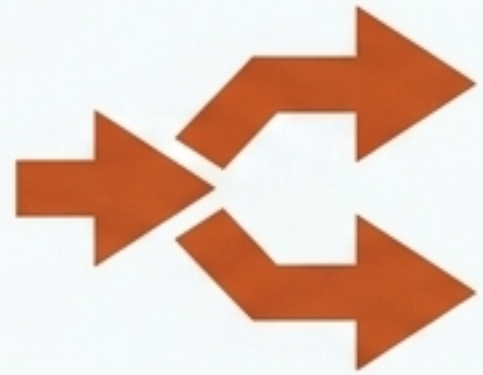


**Tipo:** `n8n-nodes-base.if`

**Propósito:** Comprobar que el servidor SMTP procesó el correo con éxito. Si falla (por ejemplo, si el correo rebotó o la conexión falló), pasa al canal de Telegram.



# Nodo 11: “Tiene Telegram?” (Bifurcación / Router IF)



**Tipo:** `n8n-nodes-base.if`

**Propósito:** Comprobar si se dispone del identificador numérico de Telegram del usuario (`cliente_telegram_id`) para enviar el aviso por el Bot de Telegram.

**Configuración Clave:**

**Regla:** Si `cliente_telegram_id` no está vacío, va a TRUE (Telegram Send).



# Nodo 12: 'Telegram Send' (Envío mediante Bot de Telegram)



**Tipo:** `n8n-nodes-base.telegram`

**Propósito:** Enviar el recordatorio de cobranza al chat privado del cliente en Telegram a través de nuestro bot dedicado.

**Configuración Clave:**

**Credentials:** Usa la API Token del Bot de Telegram (creado con BotFather).

**Chat ID:** `{{ $json.cliente_telegram_id }}` (Identificador numérico único del usuario).

**Text:** `{{ $('Generar Mensaje con IA').item.json.text }}`



# Nodo 13: 'Registrar Envío' (Actualización de Base de Datos MySQL)



**Tipo:** n8n-nodes-base.mysql

**Propósito:** Modificar el estado del cliente en la base de datos MariaDB para registrar la fecha y hora exacta del envío del último recordatorio. Esto evita enviar recordatorios duplicados y mantiene el historial de cartera actualizado.

Operation: executeQuery

```
SQL Query: UPDATE facturas SET fecha_ultimo_recordatorio = NOW()  
WHERE id = {{ $('Consultar Facturas Vencidas').item.json.id }};
```

## Explicación de la Expresión:

NOW(): Función SQL que guarda la fecha y hora actuales.

{{ \$('Consultar Facturas Vencidas').item.json.id }}:  
Apunta explícitamente al identificador (id) del cliente procesado originalmente por el primer nodo del flujo.



# Matriz de Valor Arquitectónico: Por qué esta arquitectura es destacable



## Resiliencia Multicanal (Cascada/Fallbacks)

El sistema no asume que un solo canal funcionará siempre. Si el cliente no tiene WhatsApp, o el mensaje falla, el sistema lo detecta y lo envía por Email. Si el Email no está disponible, lo envía por Telegram.

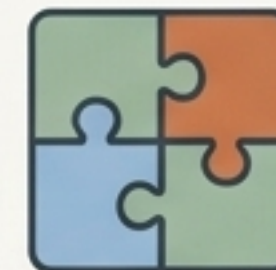
**Impacto en Infraestructura:** Tolerancia a fallos y alta disponibilidad en la capa de aplicación.



## Inteligencia Artificial Local (Ollama)

La generación de mensajes se realiza con un LLM corriendo localmente en el servidor. Esto significa costo \$0 por token y total privacidad de los datos de los clientes.

**Impacto en Infraestructura:** Soberanía de datos, eficiencia computacional on-premise y privacidad, ya que la información nunca sale de la VPS.



## Desarrollo Visual e Integrado

Combina base de datos relacional (MariaDB), microservicios de mensajería y LLMs en un flujo lógico visual de alto nivel, reduciendo drásticamente el código duro.

**Impacto en Infraestructura:** Orquestación eficiente de microservicios y mantenibilidad del sistema.